T22

Third-Party Tools

Technical Intermediate

The essential complementary tools in the ecosystem

Cost tracking

The tools in this card are adjacent layers: they observe, configure, or coordinate sessions. They do not replace the runtime that owns the agent loop. Use the Agent Harness Map to choose a runtime or orchestrator.

ccusage: reads local session usage records and estimates costs. Review its network behavior and pricing source; local input alone does not establish zero network access.

```
npx ccusage daily --last 1 # Latest day
ccusage daily       # Daily report
ccusage blocks --live # Real-time monitoring (5h windows)
ccusage daily --breakdown # Breakdown by model
```

ccburn: Python alternative with burn-rate charts in the terminal. Useful for visualizing consumption relative to billing limits, within a Python ecosystem.

```
pip install ccburn
ccburn
```

Dashboard & monitoring

ccboard: TUI and web interface for monitoring sessions, costs, and statistics in real time.

```
cargo install ccboard
ccboard         # TUI interface
ccboard --web   # Web UI on localhost:3000
```

Social sharing & leaderboard

Straude: social dashboard to track and share consumption stats. Pushes daily metrics (cost, tokens, models) to a public leaderboard. Note: sends machine hostname and a device UUID to Straude servers. Use `--dry-run` before the first push.

```
npx straude@latest   # Auth + sync in one command
straude push --dry-run  # Preview without sending
straude status       # Streak, global rank, totals
```

viberank: leaderboard covering more tools (Claude Code, Codex, Gemini CLI, OpenCode, and more). Sends aggregate totals only, no hostname or device UUID. Publishes its dataset via a free JSON API under CC BY 4.0.

```
npx viberank-cli     # Submit ccusage data
```

Token optimization

RTK (Rust Token Killer): CLI proxy that filters outputs before they enter the context. Savings depend on the command and output; its estimates do not establish a corresponding reduction in the API bill. See card T20 for details.

```
brew install rtk
rtk init          # Setup with automatic hook
```

Parallel orchestration

Conductor: macOS application for orchestrating multiple Claude Code instances in parallel via git worktrees. Integrated interface: diff viewer, CI/GitHub Actions, Linear, PR workflow.

Install from conductor.build (macOS only as of March 2026). Typical flow: create one workspace per feature from a GitHub or Linear issue, let the agents run, use “Next Workspace” to handle only those waiting for input.

Recommendation table

Profile	Recommended tools
Solo developer	ccusage + claude-code-viewer
Small team	ccusage + Conductor or multiclaude
Multi-agents	Conductor or Toad
Python ecosystem	ccburn + Claude Chic

Known gaps

No visual editor for `.claude/skills/` skill files. No unified dashboard combining config, sessions, costs, and MCP. A dedicated TUI exists for MCP servers (`claude-code-config`, `pip install claude-code-config`), but it stays limited to the `~/.claude.json` scope: no `settings.json`, hooks, or commands.

These projects evolve independently from Claude Code. Check the repository, permissions, and targeted release before installation; the glossary distinguishes a runtime, orchestrator, protocol, and adjacent tool.